

Carry Depth: A Circuit-Independent Coordinate of ARX, and the One-Round Advantage Cliff

Peter Hollows

June 22, 2026

Abstract

The hardness of SHA-256d mining reduces, by the accounting of the first paper in this series [1], to a single question: whether the round function admits a local shortcut that resolves the output without resolving the carries on the path. We give that question a coordinate. The carry chain of modular addition is the only operation in SHA-256 whose nonlinearity couples bit positions, so we define *carry depth*, the number of modular-addition layers on the dependency path between a controllable input and a target output, and compute it exactly for SHA-256d by static dataflow analysis: 386 layers. We then measure how far local structure actually reaches. A hand-built carry-aware score predicts a downstream output byte with selection advantage 0.886 one adder layer away, and that advantage falls below detection (≤ 0.0011 , consistent with zero) one round deeper: a cliff, with no geometric tail. We account for the cliff through the carry chain’s spectral structure (Holte’s matrix, contraction rate $\frac{1}{2}$ fixed by the base), separating the geometric per-bit decay from the per-round mixing that produces the reset, and we corroborate the cost reading with an exact-solver boundary, where Z3 nonce recovery cliffs by more than three orders of magnitude at round 11. The 386-layer separation is therefore wide: the strongest local attack we tested expires about 100 rounds before the output.

1 Introduction

The companion paper [1] bounds Bitcoin mining from two sides. The per-candidate work is confined to three known sharing regions, and the candidate count is locked at 2^D in the ideal model and reduced, in the standard model, to a SHA-256 distinguisher. That accounting localizes the entire residual to one object: a global algebraic shortcut that reads the output without resolving the carries on the path. It does not say whether such a shortcut exists. This paper studies that question structurally.

We make three contributions. First, we isolate the carry chain of modular addition as the sole position-coupling nonlinearity of SHA-256, so that the difficulty of local prediction is exactly the difficulty of resolving carries (Sections 2–3). Second, we turn this into *carry depth*, an exact, implementation-invariant coordinate counting carry layers along the input-to-output dataflow, and compute it at 386 for SHA-256d (Sections 4–5). Third, we measure the reach of local structure on real reduced SHA-256 and find a one-round cliff (Section 6), explain it through the carry chain’s spectral structure (Section 7), and corroborate the cost interpretation with an exact-solver boundary (Section 8). The separation between the 386-layer depth and the one-layer reach is what makes the residual of [1] a thin target rather than a wide one.

2 Carries are the entire nonlinearity of addition

For $c = a + b \bmod 2^n$, define the carry vector γ by $\gamma_0 = 0$ and $\gamma_{i+1} = \text{MAJ}(a_i, b_i, \gamma_i)$. Then bit by bit,

$$c_i = a_i \oplus b_i \oplus \gamma_i, \quad \text{i.e.} \quad a + b = a \oplus b \oplus \gamma(a, b).$$

The $a \oplus b$ term is GF(2)-linear, and *all* nonlinear, cross-position behaviour of addition is carried by γ . If γ were known, addition would be pure XOR, that is, linear. Fixing every nonlinear-gate output (each carry bit, each Ch or Maj term) collapses SHA-256d to a GF(2)-affine map, and finding an input that hits the target becomes a solvable linear system. Hardness is the cost of resolving carries.

3 Carries are the only position-coupling nonlinearity

Classify each SHA-256 operation by how it moves information:

operation	linear over GF(2)?	couples bit positions?
rotations, shifts, XOR, $\Sigma_0, \Sigma_1, \sigma_0, \sigma_1$	yes	linearly
Ch, Maj	no (degree 2)	no; bit-local (out_i sees only in_i)
modular +	no	yes ; out_i depends on all $\text{in}_{<i}$

The modular adder is the unique operation whose own nonlinearity couples different bit positions: a high output bit depends on all lower input bits, through the carry chain. (Ch and Maj are nonlinear but bit-local; they inject no carry.) The round function is a bijection on the state (SHACAL-2, invertible given the message word), so the adder destroys no information. What it removes is *locality*: the high bits of a sum cannot be read without resolving the entire low-side carry chain.

4 The carry Markov chain and Holte's matrix

Exploitable structure contracts with carry distance rather than merely becoming unknown, and under an idealizing assumption the contraction has a precise rate. For a single adder with operand bits a_i, b_i *uniform and independent*, the carry is a two-state Markov chain $\gamma_{i+1} = \text{MAJ}(a_i, b_i, \gamma_i)$ with

$$\Pr[\gamma_{i+1}=1 \mid \gamma_i=0] = \frac{1}{4}, \quad \Pr[\gamma_{i+1}=1 \mid \gamma_i=1] = \frac{3}{4},$$

that is, transition matrix $\begin{pmatrix} 3/4 & 1/4 \\ 1/4 & 3/4 \end{pmatrix}$, stationary $(\frac{1}{2}, \frac{1}{2})$, second eigenvalue $\frac{1}{2}$. Starting from the known $\gamma_0 = 0$, the bias decays geometrically,

$$\Pr[\gamma_i = 0] = \frac{1}{2} + 2^{-(i+1)}.$$

This is Holte's carry matrix [2]. In base 2, summing k operands gives a k -state carry chain on $\{0, \dots, k-1\}$ whose eigenvalues are exactly $\{2^{-j} : 0 \leq j < k\}$ (the same matrix governs the base-2 riffle shuffle [3]), so the second eigenvalue, and hence the per-bit contraction rate, is $\frac{1}{2}$ *independent of the operand count*. Extra operands introduce only faster-decaying modes; the gap is fixed by the base, never by k . The carry-in becomes a fair coin about i positions above the anchor, and chaining adders composes these contractions, which suggests that exploitable advantage decays as roughly $2^{-\Theta(\text{carry depth})}$, where carry depth counts the modular-add layers between the bits an attacker controls and the target bit. The uniform-independent assumption does not hold inside SHA-256, where state words are structured and correlated, so this is a heuristic and a prediction to test rather than a theorem; Section 6 measures the actual decay. This carry recurrence is precisely the

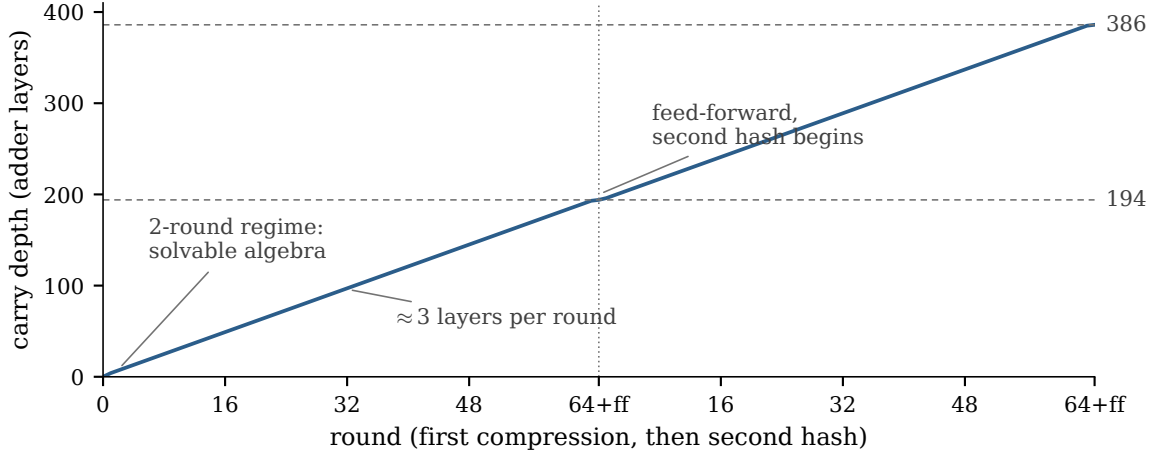


Figure 1: Carry depth along the SHA-256d dataflow, from the minimum-depth accounting of `repro/carry_depth.py`. The 64 rounds of the first compression lift the state by about three adder layers each; the feed-forwards and the second hash add full-width carry resets, reaching 386 layers from controllable input to output.

finite state of the addition viewed as an S-function [4], the modern framework for ARX differential analysis [5], in which difference propagation through modular addition is governed entirely by the carry state machine.

5 Depth accounting: SHA-256d is 386 layers

Carry depth is the structural coordinate of the proof-of-work function space: a two-round reduced hash is solvable because its input-to-target depth is $\mathcal{O}(1)$, whereas full SHA-256d sits hundreds of layers past that. We compute the coordinate exactly by static dataflow analysis (no hash evaluation; see `repro/carry_depth.py`). We treat the block-1 message words as the source inputs (the miner controls them through the nonce, the extranonce-driven Merkle root, the version, and the timestamp), use the minimum-depth associative tree for each multi-operand sum (conservative, since it credits the attacker with the shallowest legal circuit), and count Ch and Maj as depth 0 (they inject no carry, so the figure is a *floor* on the nonlinear separation, ignoring their additional bit-local nonlinearity). The figure is convention-dependent: a sequential or nonce-only accounting gives a somewhat larger number, so the minimum-depth value below is the conservative choice.

quantity	value
one compression (64 rounds + feed-forward)	194 adder layers
SHA-256d input→output adder depth (min-tree)	386
per-round increment	≈ 3 layers
modular adds on the path	1200
carry bits (≈ 31 per 32-bit add)	≈ 37,200

The two feed-forwards and the second hash each act as an independent carry reset (a fresh full-width operand), which is why they contribute more separation than their round count alone would suggest; Figure 1 traces the profile.

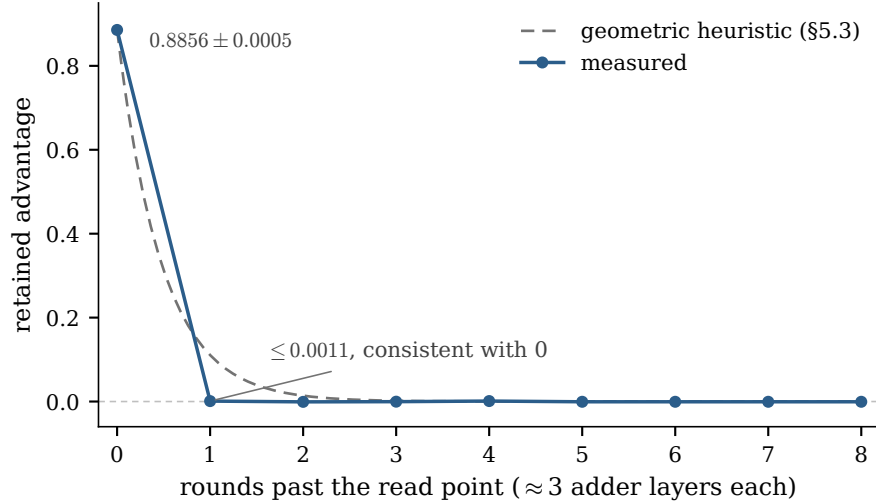


Figure 2: The anchor sweep: selection advantage of the carry-aware score read j rounds past the read point, against the geometric decay suggested by the carry Markov heuristic of Section 4. The measurement returns a cliff rather than a slope; past one round every value sits within the $\approx 10^{-3}$ detection limit of zero.

6 The anchor sweep: a one-round cliff

The depth fixes the exponent’s base count (386); the margin is $386 \cdot \log_2(1/\rho)$, where ρ is the advantage retained per adder layer. We measure ρ directly on real reduced SHA-256 (see `repro/anchor_sweep.py`). The carry-aware score $(T_1 \gg 24) + (T_2 \gg 24)$, read at an interior round, strongly predicts that the top byte of `state[0]` is small one layer downstream, and we measure how that control decays as the observed target moves deeper. Over 24 message stems (header-prefix stand-ins; see the code for the simplified setup) and 2^{22} candidates each (standard error on the mean advantage $\approx 10^{-3}$):

depth from read point	retained advantage
1 adder layer (same round)	0.8856 ± 0.0005
one round deeper (≈ 3 layers)	≤ 0.0011 (consistent with 0)

The advantage *cliffs* (Figure 2): it falls from 0.886 to undetectable within a single round. There is no geometric tail to fit; $\rho \approx 0$, a one-layer reset. The mechanism is explicit: at the next round `state[0]` enters Σ_0 , whose rotations scatter the controlled top byte across all bit positions, and it is then re-randomized by a fresh carry chain, so the controlled structure is delocalized off the top-byte observable in one round. The information is preserved (the round is invertible); the accessibility is destroyed.

The combined statement. The target output sits 386 adder layers from the controllable header inputs, while the strongest local advantage we measured has reach of about one layer and is dead within a single round (≈ 3 layers). The margin therefore does not rest on a thin ρ^{386} extrapolation; the advantage is already gone roughly 100 rounds before the output. We deliberately do not compound the measured one-sided bound $\rho_{\text{round}} \leq 0.0011$ (a 2σ upper bound on the deeper-round mean advantage) across the remaining rounds: that bound is a detection limit consistent with $\rho = 0$,

not a measured decay rate, and multiplying it would manufacture a precise-looking exponent from a null result. The honest statement needs no extrapolation, since the advantage is already below detection after a single round and SHA-256d applies 128.

7 Two decay scales

The Markov heuristic of Section 4 predicts a geometric slope, $2^{-\Theta(\text{depth})}$, while the anchor sweep returns a cliff. These are consistent once two different axes are separated.

The geometric 2^{-d} decay lives on the *bit-position* axis within a single addition: it is the carry chain mixing along its own length, the contraction whose rate is the second eigenvalue $\frac{1}{2}$. This is the quantity that linear and differential cryptanalysis of one ARX step sees, and it is a genuine slope.

The *round* axis is different. One SHA-256 round is not one step of that chain. The controlled word enters Σ_0 , three rotations combined by XOR, which scatters its top byte across all bit positions, and it is then fed as one addend into a fresh full-width carry chain with independent operands. That is a full mixing step, not a single contraction, so the retained advantage from a fixed read point collapses to the noise floor in one round rather than decaying as a per-round ρ . The naive reading conflates one carry step with one round; the cliff is what a full per-round mixing time looks like, and the 386-layer depth is the count of such steps to the output.

8 The SAT-solver boundary at eleven rounds

Carry depth predicts a solving cost as well as a statistical reach. If reading the target requires resolving about three carry layers per round, a complete solver should see its work grow geometrically with the round count. We measure this with an exact SMT solver, Z3 [11]. The instance fixes the nonce as the single 32-bit unknown W_3 , takes the state after r rounds of the correct round function for a hidden true nonce, and asks for a nonce reaching it; it is satisfiable by construction, and the returned witness is verified against `hashlib`.

Across rounds the wall-clock cost (two independent seeds, external hard deadlines) is 0.08 s at 8 rounds, 6.3 s at 10, and a timeout beyond 7200 s at 11, 12, and 13, with 14 rounds and above undecided at the budget. The cost grows about $80\times$ per two rounds through round 10, then jumps more than three orders of magnitude between round 10 and round 11: a one-round cliff at the eighth nonce-dependent round. Folklore held that the solver “stops returning somewhere between 12 and 64 rounds”; the single archived 12-round solve used a round function with a Σ_1 miscomputation that weakens it, and on the correct circuit 12 rounds does not return in 6.4 hours at about 50 threads. The geometric growth and the cliff are the cost-side image of the same carry composition the anchor sweep sees statistically: about three adder layers per round, a conflict space that grows like $2^{\Theta(\text{depth})}$.

9 Relation to circuit-complexity measures

Carry depth should not be conflated with the circuit measures it superficially resembles. The closest prior notion is the carry-truncation depth of Field and Jones [6], who independently identify the carry as the sole nonlinearity of ARX; their depth is a local approximation parameter on a single addition, whereas carry depth is an exact, global count of carry layers along the input-to-output dataflow of the whole function. It is likewise distinct from the FHE/MPC notion of *multiplicative* (AND) *depth*. The standard Bristol-Fashion circuit for one SHA-256 compression has AND-depth 1607 [7, 8], far above our 386, because AND-depth counts every nonlinear gate on the path, including

the bit-local Ch and Maj gates and every gate of a ripple-carry adder, while carry depth counts only carry layers and assigns Ch and Maj depth 0. AND-depth is a property of a chosen circuit: the same modular addition has linear AND-depth as ripple-carry but logarithmic depth as a parallel-prefix adder [9], since the carry-merge operator is associative, and published heuristics shrink a 128-bit adder from AND-depth 255 to 9 without altering its logic [8]. Carry depth counts the logical carry-dependency layers and is invariant to the realization; it is a coordinate of the function, not of an implementation. Nor is it multiplicative *complexity*, a gate *count* (an n -bit adder needs exactly $n-1$ AND gates [10]) carrying no notion of depth.

10 Reproduction and conclusion

Two quantitative results are reproducible from self-contained code, built on a vendored minimal SHA-256 verified bit-exact against `hashlib`. `carry_depth.py` computes the 386-layer accounting by static dataflow analysis (pure Python, no dependencies, instant). `anchor_sweep.py` measures the cliff (0.886 at depth one, consistent with 0 one round deeper); the full run uses `numpy/cupy`, and a `--pure` standard-library path reproduces the cliff and a `hashlib` parity check on any interpreter. The SAT boundary is reproducible from the Z3 nonce-recovery sweep.

The carry as the sole nonlinearity of ARX is known [6, 5, 4]. The contribution here is to turn it into carry depth, an explicit, circuit-independent coordinate of the proof-of-work function space, to compute it exactly at 386 for SHA-256d, and to measure that the strongest local advantage has reach of about one layer. The structural separation is large, the measured reach is one layer, and the residual isolated by the companion paper [1] is therefore a thin target: any attack that beats brute force must reach across 386 carry layers that the natural local methods cross by one.

References

- [1] P. Hollows. *The Amortization Envelope: Where a SHA-256d Mining Advantage Can and Cannot Live*. Carry-Adder Wall series I, 2026.
- [2] J. M. Holte. *Carries, Combinatorics, and an Amazing Matrix*. American Mathematical Monthly 104(2), 1997.
- [3] P. Diaconis, J. Fulman. *Carries, Shuffling, and an Amazing Matrix*. American Mathematical Monthly 116(9), 2009.
- [4] N. Mouha, V. Velichkov, C. De Cannière, B. Preneel. *The Differential Analysis of S-functions*. SAC 2010.
- [5] H. Lipmaa, S. Moriai. *Efficient Algorithms for Computing Differential Properties of Addition*. FSE 2001.
- [6] R. E. Field, B. C. Jones. *Using Carry-Truncated Addition to Analyze Add-Rotate-Xor Hash Algorithms*. 2013, [arXiv:1303.4448](https://arxiv.org/abs/1303.4448).
- [7] N. P. Smart et al. ‘Bristol Fashion’ MPC Circuits. <https://nigelsmart.github.io/MPC-Circuits/>.
- [8] P. Aubry, S. Carpov, R. Sirdey. *Faster Homomorphic Encryption is not Enough: Improved Heuristic for Multiplicative Depth Minimization of Boolean Circuits*. CT-RSA 2020.

- [9] T. G. Draper, S. A. Kutin, E. M. Rains, K. M. Svore. *A Logarithmic-Depth Quantum Carry-Lookahead Adder*. Quantum Inf. Comput. 6(4), 2006.
- [10] J. Boyar, R. Peralta. *Tight Bounds for the Multiplicative Complexity of Symmetric Functions*. Theoretical Computer Science 396, 2008.
- [11] L. de Moura, N. Bjørner. *Z3: An Efficient SMT Solver*. TACAS 2008.